

**ACTIVIDAD DE APRENDIZAJE UNIDAD 2: SISTEMA DE RESTAURANTE
CON LISTA, COLA Y PILA**

ESTUDIANTE:

Rey David Borrego Pérez

ASIGNATURA:

ESTRUCTURA DE DATOS

DOCENTE:

JHON ARRIETA ARRIETA

PROGRAMA:

INGENIERÍA DE SOFTWARE

UNIVERSIDAD DE CARTAGENA

2026-1

1. Introducción

La Unidad 2 de Estructura de Datos se centra en la construcción manual de estructuras lineales, especialmente lista, cola y pila, usando nodos enlazados. En este informe se desarrolla el ejercicio asignado número 4, correspondiente al caso Restaurante, cuya entidad principal es Pedido y cuyo identificador es el número de pedido. La aplicación simula el flujo de atención de un restaurante: los pedidos se registran, pasan a una cola de pendientes, se procesan en orden de llegada y quedan almacenados en una pila de historial.

El propósito técnico del trabajo es demostrar que una solución de consola en Java puede resolver un problema real mediante estructuras personalizadas. A diferencia de usar colecciones ya implementadas por el lenguaje, en esta actividad se crea explícitamente la lógica de nodos, enlaces, recorridos, inserciones, eliminaciones y consultas, lo que permite comprender el funcionamiento interno de las estructuras de datos lineales (Cormen et al., 2022; Goodrich et al., 2014).

El documento mantiene una estructura académica similar al informe de la Unidad 1: introducción, objetivos, justificación, desarrollo técnico, enlace de repositorio, enlace de video, conclusiones y referencias. Además, se agregan los logros de aprendizaje y la explicación del flujo del sistema solicitados por el protocolo de la actividad.

2. Objetivos

Objetivo General

Implementar un sistema de gestión de pedidos para restaurante en Java, utilizando estructuras de datos personalizadas tipo lista, cola y pila, con el fin de comprender el registro, procesamiento, búsqueda, cancelación y deshacer de operaciones mediante nodos enlazados.

Objetivos Específicos

- Diseñar la clase Pedido con atributos, constructor, métodos de acceso, toString() y equals() basado en el número de pedido.
- Implementar una Lista personalizada para registrar todos los pedidos ingresados al sistema.
- Implementar una Cola personalizada para administrar los pedidos pendientes bajo el principio FIFO.
- Implementar una Pila personalizada para guardar el historial de pedidos procesados y permitir deshacer el último procesamiento.
- Construir un menú de consola que permita registrar, consultar, procesar, cancelar y contar pedidos.
- Documentar el funcionamiento del sistema, los fragmentos relevantes del código y las evidencias de ejecución.

3. Justificación

El aprendizaje de estructuras de datos personalizadas es fundamental para comprender cómo se organiza y se recorre la información en memoria. Aunque Java ofrece colecciones listas para usar, construir una lista, una cola y una pila desde cero permite observar directamente cómo un nodo almacena un dato y una referencia hacia otros nodos. Esta comprensión es necesaria para analizar problemas de rendimiento, errores de referencia y flujo interno de datos en programas más complejos (Sedgewick & Wayne, 2011).

El caso de restaurante es adecuado para aplicar estas estructuras porque representa un proceso cotidiano: todos los pedidos deben quedar registrados, los pendientes deben atenderse en orden de llegada y los procesados deben conservarse como historial. La lista permite conservar la trazabilidad general, la cola respeta el orden de atención y la pila facilita recuperar el último procesamiento en caso de error.

4. Desarrollo

4.1 Logros de aprendizaje

Durante el desarrollo de la actividad se fortaleció la comprensión de las estructuras lineales personalizadas. La lista se entendió como una secuencia de nodos que permite registrar y recorrer todos los elementos; la cola se comprendió como una estructura FIFO, donde el primer pedido ingresado es el primero en procesarse; y la pila se aplicó como una estructura LIFO, donde el último pedido procesado es el primero que puede retirarse para deshacer la operación.

También se reforzaron conceptos de programación orientada a objetos, como encapsulamiento, creación de clases, constructores, getters, setters, sobrescritura de toString() y comparación mediante equals(). En la clase Pedido, el número de pedido se usa como identificador principal para evitar duplicados y realizar búsquedas coherentes.

Una dificultad importante fue comprender que cancelar un pedido pendiente no debía resolverse eliminando directamente desde el centro de la cola. Por esa razón se utilizó una cola auxiliar, que permite reconstruir la cola original excluyendo el pedido cancelado. Esta decisión respeta el comportamiento natural de la cola y evita romper el orden FIFO.

4.2 Descripción funcional del sistema

El sistema permite administrar pedidos de un restaurante desde consola. Cada pedido contiene número de pedido, cliente, descripción, total y estado. Cuando se registra un pedido, queda guardado en la lista general y entra a la cola de pendientes. Luego, al procesar el siguiente pedido, se desencola el primer pendiente, se cambia su estado a PROCESADO y se apila en el historial.

El usuario interactúa mediante un menú que permite registrar pedidos, ver todos los pedidos, consultar pendientes, procesar el siguiente, revisar el historial, buscar por número, cancelar un pendiente, deshacer el último procesamiento, ver cantidades y salir del sistema.

Tabla 1. Relación entre operaciones del restaurante y estructuras personalizadas.

Operación	Estructura usada	Resultado en el sistema
Registrar pedido	Lista + Cola	El pedido queda guardado y pendiente por procesar.
Procesar pedido	Cola + Pila	El primer pendiente pasa al historial como procesado.
Buscar pedido	Lista	Se recorre la lista para encontrar el número solicitado.
Cancelar pendiente	Cola auxiliar	Se reconstruye la cola sin el pedido cancelado.
Deshacer procesamiento	Pila + Cola	El último procesado vuelve a la cola de pendientes.

4.3 Entidad principal: Pedido

La entidad Pedido representa el elemento central del caso Restaurante. Se diseñó con cinco atributos: numeroPedido, cliente, descripcion, total y estado. El método equals() compara por número de pedido, porque ese dato identifica de forma única cada solicitud dentro del sistema.

```
public class Pedido {
    private String numeroPedido;
    private String cliente;
    private String descripcion;
    private double total;
    private String estado;

    @Override
```

```

public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Pedido)) return false;
    Pedido otro = (Pedido) obj;
    return numeroPedido.equalsIgnoreCase(otro.numeroPedido);
}
}

```

4.4 Componentes desarrollados

Tabla 2. Clases implementadas en la Unidad 2.

Clase	Responsabilidad principal
Nodo	Guardar el dato y las referencias necesarias para enlazar los elementos.
Lista	Registrar todos los pedidos y permitir recorridos hacia adelante y hacia atrás.
Cola	Administrar los pedidos pendientes bajo el principio FIFO.
Pila	Guardar el historial de pedidos procesados bajo el principio LIFO.
Pedido	Modelar la información funcional del pedido del restaurante.
SistemaRestaurante	Ejecutar el menú, coordinar estructuras y controlar el flujo del sistema.

4.5 Explicación de las estructuras de datos utilizadas

La Lista personalizada actúa como registro general. Al insertar un pedido, se crea un nodo que almacena el objeto y se enlaza con los nodos existentes. Esta estructura resuelve el requerimiento de conservar todos los pedidos, incluso si luego se procesan o se cancelan.

La Cola personalizada representa los pedidos pendientes. Al usar FIFO, el pedido que ingresa primero se procesa primero. En el sistema, encolar significa agregar el pedido al final de la cola y desencolar significa retirar el pedido ubicado al frente.

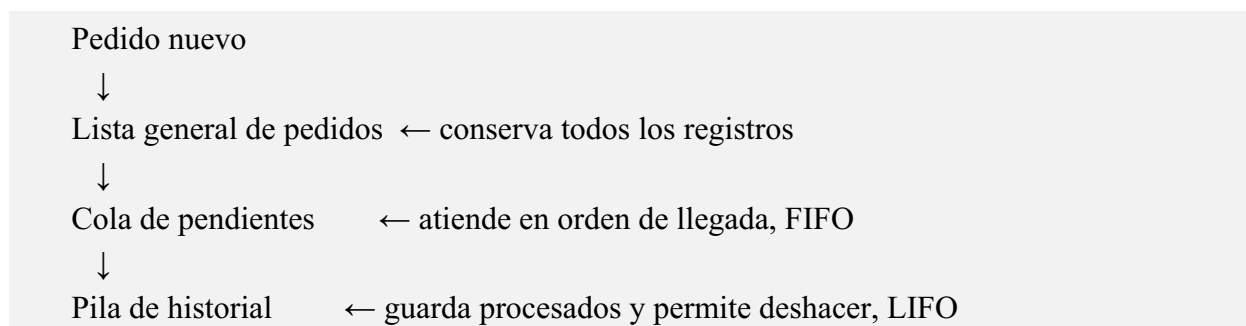
La Pila personalizada se utiliza como historial de procesados. Su principio LIFO permite que el último pedido procesado sea el primero disponible para deshacer. Por eso, si el usuario selecciona deshacer, se desapila el último elemento y se devuelve a la cola de pendientes.

4.6 Flujo del sistema

Tabla 3. Flujo general del sistema de restaurante.

Paso	Descripción
1	El usuario registra un pedido con número, cliente, descripción y total.
2	El sistema valida que no exista otro pedido con el mismo número.
3	El pedido se agrega a la lista general y se encola como pendiente.
4	Al procesar, se desencola el primer pedido pendiente.
5	El pedido cambia a estado PROCESADO y se apila en el historial.
6	La búsqueda recorre la lista general comparando el número de pedido.
7	La cancelación usa una cola auxiliar para excluir el pedido pendiente.
8	La operación deshacer desapila el último procesado y lo encola nuevamente.

Representación simplificada del flujo:



4.7 Fragmentos importantes del código

El siguiente fragmento muestra el registro del pedido. La misma entidad se agrega a la lista general y a la cola de pendientes, cumpliendo el flujo obligatorio de la actividad.


```
Pedido pedido = new Pedido(numero, cliente, descripcion, total, "PENDIENTE");  
listaGeneral.agregar(pedido);  
colaPendientes.encolar(pedido);  
System.out.println("Pedido registrado y enviado a la cola de pendientes.");
```

El procesamiento respeta FIFO porque toma el primer pedido de la cola; luego el pedido se marca como procesado y se apila en el historial.

```
Pedido procesado = (Pedido) colaPendientes.desencolar();  
if (procesado == null) {  
    System.out.println("No hay pedidos pendientes por procesar.");  
    return;  
}  
procesado.setEstado("PROCESADO");  
historialProcesados.apilar(procesado);
```

4.8 Evidencia de ejecución

Evidencia de ejecución por consola

```

===== SISTEMA DE PEDIDOS - RESTAURANTE =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por código
7. Cancelar elemento pendiente
8. Deshacer último procesamiento
9. Ver cantidad de elementos
10. Salir
Seleccione una opción:
--- Registrar pedido ---
Número de pedido: Nombre del cliente: Descripción del pedido: Total del pedido: Pedido registrado y enviado a

===== SISTEMA DE PEDIDOS - RESTAURANTE =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por código
7. Cancelar elemento pendiente
8. Deshacer último procesamiento
9. Ver cantidad de elementos
10. Salir
Seleccione una opción:
--- Registrar pedido ---
Número de pedido: Nombre del cliente: Descripción del pedido: Total del pedido: Pedido registrado y enviado a

===== SISTEMA DE PEDIDOS - RESTAURANTE =====
1. Registrar elemento
2. Ver todos los elementos registrados
3. Ver elementos pendientes
4. Procesar siguiente elemento
5. Ver historial de elementos procesados
6. Buscar elemento por código
7. Cancelar elemento pendiente
8. Deshacer último procesamiento
9. Ver cantidad de elementos
10. Salir

```

Figura 1. Ejecución del sistema de restaurante con lista, cola y pila.

5. Repositorio Público de GitHub

El código fuente debe publicarse en un repositorio público de GitHub. Para cumplir la exigencia de evidencia de construcción, el repositorio debe mostrar commits progresivos, mensajes descriptivos y evolución real del desarrollo. En este espacio se debe pegar el enlace definitivo después de subir el proyecto:

https://github.com/ReyDavid07/Actividad_de_AprendizajeU2_ED_UDEC

6. Conclusiones

La actividad permitió comprobar que las estructuras de datos lineales resuelven problemas reales cuando se seleccionan según el flujo del negocio. En el restaurante, la lista conserva todos los pedidos, la cola organiza la atención en orden de llegada y la pila permite consultar o revertir el último procesamiento.

Construir las estructuras manualmente ayudó a comprender el papel de los nodos y las referencias. Esta práctica es valiosa porque muestra lo que normalmente permanece oculto cuando se usan colecciones ya implementadas por Java.

El sistema también fortaleció el manejo de validaciones, modularización y separación de responsabilidades. La clase Pedido modela los datos, las estructuras administran el almacenamiento y SistemaRestaurante coordina el flujo de interacción con el usuario.

7. Referencias

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley Professional.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Java (6th ed.). Wiley.

Oracle. (n.d.). Java Platform, Standard Edition API specification.

<https://docs.oracle.com/en/java/javase/>

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.